

Plateforme intelligente de pilotage et d'analyse décisionnelle du CIAT

Journal des prompts

ISEP Amadou Trawaré de Diamniadio · Centre d'Innovation, d'Application et de Transfert
Évaluation pratique en intelligence artificielle générative · Projet 3 · document généré le 03/11/2026

Journal des prompts – CIAT VISION

Livrable n° 8. Chaque entrée consigne le prompt envoyé à l'IA, le résultat obtenu et les corrections apportées.

Entrée 1 – Phase 1 : cahier des charges

Date : 02/11/2026 **Outil :** Claude Code (Claude Opus 5.5)

Prompt : prompt de cadrage complet « PROJET : CIAT VISION » (rôle d'équipe d'experts, contexte, objectifs, identité visuelle, technologies, 10 modules, base de données, données de démonstration, exigences de qualité, tests, livrables, méthode en 12 phases), accompagné du cahier des charges version 1.0 rédigé par l'étudiant, avec la consigne : « lire bien et faire ce qui a été demandé ».

Résultat obtenu : revue du cahier des charges 1.0 par rapport au prompt, puis version 1.1 (`docs/01_cahier_des_charges.md`).

Corrections et compléments apportés par la revue : | Constat sur la version 1.0 | Correction en 1.1 | |---|---| | Aucune définition de « projet en retard » | Définition précise et seuil paramétrable (§ 7.1) | | Aucun statut pour une candidature rejetée | Statut « Non retenu » proposé, à valider (§ 5.2, V1) | | Droits de modification non attribués (qui crée ou supprime un projet ?) | Matrice des droits par rôle (§ 4.2) | | Le prompt interdit le SQL arbitraire, mais la v1.0 ne le précisait pas | Liste fermée de fonctions d'analyse, aucun SQL généré par l'IA (§ 6.7) | | « Financement total » ambigu (accordé ou reçu ?) | Montants demandé, accordé, reçu, reste à recevoir et besoin non couvert définis (§ 5.4, § 7.2) | | « Financement obtenu » saisi dans le projet : risque d'incohérence | Champ calculé à partir des financements | | Mécanisme de remplacement des données simulées non décrit | Indicateur `is_simulated`, bandeau, purge par l'administrateur (§ 11.2) | | Texte blanc sur beige `#CA9871` : contraste 2,5 : 1, illisible | Règles de contraste vérifiées (§ 9.2) | | Pas de critères d'acceptation mesurables | Critères par module (§ 16.2) | | Planning différent des 12 phases du prompt | Planning réaligné (§ 15) | | Données personnelles non traitées | Loi n° 2008-12, minimisation, rien envoyé à l'IA (§ 8.3) |

Statut : points V1 à V5 validés (voir entrée 2) ; V6 (logo) et V7 (organisation individuelle) en attente d'éléments externes.

Entrée 2 – Phases 2 à 12 : réalisation complète

Date : 02/11/2026 **Outil :** Claude Code (Claude Opus 5.5)

Prompt : « fait tout ce qui a été demandé » (suite de l'entrée 1 : réaliser l'ensemble des phases du prompt de cadrage, les propositions V1 à V5 du cahier des charges étant acceptées).

Résultats obtenus, par phase : | Phase | Résultat | Fichiers | |---|---|---| | 2 | Architecture 3 couches, 7 diagrammes UML (Mermaid) | `docs/02_architecture.md` | | 3 | 16 tables, contraintes CHECK, index, migration Alembic, dictionnaire généré depuis le code | `backend/app/models.py`, `backend/alembic/`, `docs/03_base_de_donnees.md` | | 4 | 23 maquettes haute fidélité (ordinateur 1 440 px, téléphone 390 px) |

`docs/04_maquettes.md`, `docs/maquettes/` | | 5 | Squelettes FastAPI + React/Vite/Tailwind, PostgreSQL 16 | `backend/`, `frontend/` | | 6 | JWT, bcrypt, rôles, blocage après échecs, journal d'audit, administration des comptes | `security.py`, `routers/auth.py`, `routers/admin.py` | | 7 | Projets (cycle de vie, évaluations, jalons), cohortes | `services/projects.py`, `routers/projects.py`, `routers/reference.py` | | 8 | Moteur unique d'indicateurs, 12 graphiques, financements | `services/kpi.py`, `components/charts.tsx` | | 9 | CIAT AI Analyst : appel d'outils sur 7 fonctions autorisées, mode dégradé | `services/analysis.py`, `services/assistant.py` | | 10 | 6 rapports PDF/Excel/CSV ; importation contrôlée | `services/reports.py`, `services/importer.py` | | 11 | 88 tests automatisés, parcours navigateur, 10 anomalies corrigées | `backend/tests/`, `frontend/src/test/`, `docs/05_tests_et_anomalies.md` | | 12 | Blueprint Render, guides, rapport, scénario vidéo | `render.yaml`, `docs/06` à `docs/09` |

Corrections apportées en cours de route (détail : `docs/05_tests_et_anomalies.md` § 4) : - Docker indisponible sur le poste → PostgreSQL 16 portable (A1). - Données simulées irréalistes puis historiques incohérents → générateur corrigé et test de cohérence ajouté (A2, A6). - Plantage de la page de l'assistant détecté par le parcours navigateur, cause `scrollIntoView()` renvoyant une promesse dans un `useEffect` → corrigé (A5). - Tailwind 4 (`@apply` sur classe personnalisée), mise en page PDF, affichage mobile, messages anglais, taille du paquet JavaScript (A3, A4, A7, A8, A10).

Décision prise pendant la réalisation : la suspension n'est possible qu'à partir de « En incubation » (cahier des charges v1.2), pour que tout projet suspendu soit un projet incubé et que le taux d'achèvement soit sans ambiguïté.

Ce qui reste à faire par le réalisateur : fournir le logo officiel CIAT VISION ; créer le dépôt GitHub et déployer sur Render (procédure dans `docs/06_guide_installation.md` § 5) ; tester le mode IA avec une clé d'API ; enregistrer la vidéo (`docs/09_scenario_video.md`) ; faire valider la réalisation individuelle par l'enseignant.

Entrée 3 – Compléments : test de charge et livrables PDF

Date : 03/11/2026 **Outil :** Claude Code (Claude Opus 5.5)

Prompt : « continue ».

Résultats obtenus : - **Test de charge à la volumétrie de la cible (1 030 projets)**, sur une base dédiée : tableau de bord affiché en 1,0 à 1,8 s dans le navigateur (cible < 2 s) ; API des indicateurs en 38 ms ; rapport PDF de 1 030 lignes en 4,8 s (cible < 10 s). Script reproductible `backend/scripts/mesurer_performances.py` ; résultats dans `docs/05_tests_et_anomalies.md` § 3 et dans le rapport. - **Versions PDF des 10 livrables** (`docs/pdf/`), avec couverture, pagination et diagrammes Mermaid dessinés ; script `scripts/generer_pdf_livrables.py`. Le rapport compte 7 pages (exigence : 5 à 10).

Corrections apportées : - Diagramme de cas d'utilisation illisible dans le PDF (bulles géantes, flèches croisées sur deux pages) → refait avec des groupes de cas et un héritage entre acteurs ; il tient sur une page. - Vue d'ensemble trop petite → orientation verticale. - Noms de tables minuscules dans les titres du dictionnaire → style corrigé.

Non réalisable sans le réalisateur : publication sur GitHub (l'outil `gh` n'est pas installé et la création d'un dépôt se fait avec le compte du réalisateur), déploiement Render, clé d'API Claude, logo officiel, vidéo.

Entrée 4 – Publication GitHub et intégration du logo officiel

Date : 03/11/2026 **Outil :** Claude Code (Claude Opus 5.5)

Prompts : « nom compte github est serigne-dev12 », puis envoi de l'image du logo officiel CIAT VISION.

Résultats obtenus : - Dépôt **public** créé et code publié : https://github.com/serigne-dev12/CIAT-VISION (choix du réalisateur : public, nom « CIAT-VISION »). - Avant publication, le mot de passe du PostgreSQL local a été retiré du code de test **et de l'historique Git** ; il est désormais lu dans `backend/.env` (non publié). - Version de Node.js fixée dans `render.yaml` (Vite 8 exige Node 20.19+). - Logo officiel intégré sans déformation : versions horizontale, compacte et complète obtenues par recadrage de l'original ; posé sur un cartouche blanc sur les fonds marron (barre latérale, en-tête des PDF) ; icône d'onglet ; couverture des PDF des livrables. Maquettes recapturées.

Entrée 5 – Logo sans fond

Date : 03/11/2026 **Outil :** Claude Code (Claude Opus 5.5)

Prompt : « le travail est bon mais on va améliorer le logo : enlève le background ».

Résultat obtenu : fond blanc du logo rendu transparent (seul le fond relié aux bords ; les blancs intérieurs du dessin, comme le verre de la loupe, sont conservés ; bords adoucis sans liseré). Script reproductible `scripts/preparer_logo.py`. Les cadres blancs ont été supprimés : le logo est posé directement sur une zone claire (haut de la barre latérale, menu mobile, page de connexion, en-tête blanc des rapports PDF).

Correction en cours de route : la première exécution du script ne rendait aucun pixel transparent ; cause : avec cette version de Pillow, une image créée par `Image.fromarray` partage la mémoire du tableau et le remplissage (`floodfill`) ne la modifiait pas. Corrigé par une copie (`.copy()`), puis seuil ajusté (fond de l'original à 253-254 et non 255) : 80 % des pixels deviennent transparents, 0,4 % semi-transparents (contours).